Practice: purity and base cases

Exercise 1. Classify these operations: square x = x*x; reading the current time; printing a line.
Solution: square is pure. Reading time and printing are effects. Passing a timestamp as an ordinary input to a calculation can leave that calculation pure.

Exercise 2. Complete a product function on a finite list.

```
productList [] = 1
productList (x:xs) = x * productList xs
```

The empty product is 1, the identity for multiplication. productList [2,3,4] returns 24. A base case of 0 would incorrectly make every finite product zero.

Exercise 3. Define factorial for nonnegative integers.

```
factorial 0 = 1
factorial n = n * factorial (n-1)
```

This definition assumes n >= 0. Under that assumption the argument decreases toward zero, and factorial 4 returns 24. Calling this definition on a negative integer does not reach the base case. A public interface should reject negatives or represent failure explicitly.

Original educational material / CC BY 4.0 / Bilal Amin

Practice: tracing and safe list handling

Exercise 4. Trace sumList [5,1].
Solution: sumList [5,1] = 5 + sumList [1] = 5 + (1 + sumList []) = 5 + (1 + 0) = 6.

Exercise 5. Return a list's first element without failing on an empty input.

```
safeHead [] = Nothing
safeHead (x:_) = Just x
```

The result is Maybe a. safeHead [] returns Nothing; safeHead [9,8] returns Just 9. The caller must handle the Nothing case.

Exercise 6. Explain the flaw in count xs = 1 + count xs.
Solution: it has no base case and passes the same input back to itself. It neither terminates on [] nor reduces a nonempty list. A correct structural recursion separates [] from _:xs and recurses only on xs.

Revision check: a decreasing argument is useful only if it actually approaches a case the definition handles. For example, repeatedly decrementing a negative integer moves away from zero.

Original educational material / CC BY 4.0 / Bilal Amin